

Roadmap a producción

Esta aplicación es una demo funcional de un agente comercial de Kavak. Actualmente está optimizada para entornos de desarrollo.

Para llevarla a producción, sería necesario realizar algunos ajustes e implementar ciertas mejoras para contar con un agente robusto y listo para operar en un entorno real.

A continuación, presento algunas sugerencias para avanzar hacia una versión productiva:

Circuit Breaker y fallback de modelo

Para llevarlo a producción implementaría un circuit breaker, que monitorea fallas repetidas o latencias anormales y “corta el circuito” del modelo principal por un tiempo, evitando más errores en cadena, y un fallback de modelo, que automáticamente redirige a un segundo modelo cuando el primero falla o responde mal. De esta forma, si OpenAI no responde o se demora, el sistema deja de usarlo y lo ruteamos al fallback, por ejemplo Gemini, para mantener la conversación activa sin que el usuario note interrupciones.

Deployment

Para orquestar los contenedores, podríamos pasar de Docker Compose a Kubernetes (o ECS). También tendría sentido añadir un API Gateway (AWS API Gateway) como capa de entrada: de ahí exponemos un único endpoint, enrutamos las llamadas y podemos aplicar autenticación y rate-limiting sin tocar cada servicio. Para servir la API, se puede combinar Uvicorn con Nginx con HTTPS en un load balancer. En cuanto a secretos y variables de entorno, si se toma una arquitectura basada en AWS se gestionan idealmente con AWS Secrets Manager, Vault u otro servicio similar.

Base de datos

En lugar de un container local, sería ideal usar un servicio autogestionado (RDS, Cloud SQL) que ofrezca backups y escalado.

Capa de caché

Para aligerar las búsquedas semánticas en PGVector, se podría introducir una capa intermedia de Redis y almacenar ahí resultados frecuentes con TTL para no ir a buscar a la DB consultas ya repetidas por los usuarios (menos consultas semánticas a la DB, más rápidas respuestas tendrá el usuario).

Procesamiento asíncrono para el sumador

Teniendo en cuenta que, tras el envío de mensajes por parte del usuario, se ejecuta un proceso de sumación de la conversación (sólo si corresponde), este proceso podría migrarse al uso de una cola de mensajes (como RabbitMQ o AWS SQS) con workers dedicados que la escuchen:

- Luego de N mensajes en una conversación, se envía un job a la cola.

- Un worker dedicado procesa los jobs y actualiza el resumen en segundo plano.

Como alternativa “más barata”, esto también podría implementarse utilizando los BackgroundTasks de FastAPI.

Debounce para los mensajes

Sugiero implementar un debounce con un delta de tiempo configurable (por ejemplo, 2 segundos) después del último mensaje recibido.

- Si llega un nuevo mensaje dentro de ese período, el temporizador se reinicia.
- Solo cuando pasan esos N segundos sin recibir nuevos mensajes, se ejecuta el ciclo del agente.

Esto permite agrupar varias interacciones en una sola llamada, reducir la carga sobre el modelo y mantener una respuesta rápida para el usuario, ya que la request original no queda bloqueada.

Autenticación y límites

Para proteger los endpoints, podríamos implementar OAuth2/JWT y definir roles o scopes. Y para evitar abusos, convendría aplicar rate-limiting (por ejemplo con SlowAPI, el API Gateway o una solución propia en Redis).

Métricas

Trackearía tiempos de respuesta, incluyendo la latencia promedio, uso de tokens, tanto de prompt consumidos como de completion generados; uso de la API, con el número de llamadas diarias y semanales; rate limits, monitorizando los errores y el tiempo de recuperación tras un bloqueo; tasa de error, midiendo el porcentaje de respuestas con errores; volumen de conversaciones, comparando las iniciadas versus las completadas y contando el número de turnos por sesión; éxito funcional, evaluando el porcentaje de respuestas que respetan el tono y rol esperados y el reconocimiento correcto de intents; y, finalmente, costos estimados, calculando el gasto por llamada según el precio por token y el costo acumulado diario o semanal.

Observabilidad

Sería útil exponer métricas que comenté anteriormente con herramientas como Prometheus (latencias, errores) y visualizarlas en Grafana. Los logs podrían centralizarse en ELK Stack o un servicio en la nube, y para trazas distribuidas usar OpenTelemetry.

CI/CD y despliegues

Podríamos montar pipelines en GitHub Actions o GitLab CI que construyan, prueben y desplieguen en cada cambio. Incluyamos tests unitarios e integración, y definamos un plan de rollback que sea rápido de ejecutar basado en casos de testeos (como implemente en

evaluator) pero versionados.

Medición del desempeño del agente

Si tuviera que evaluar cómo está funcionando el bot, lo primero que haría es mirar si realmente está ayudando a los usuarios sin tener que pasar a un humano. Es decir, me fijaría cuántas conversaciones llegan a buen puerto de forma automática (tasa de resolución de casos).

Si esa métrica baja después de un cambio, seguro es un indicio que algo se rompió.

También prestaría mucha atención a cuántas veces el bot no entiende lo que le dicen. Si responde seguido con cosas como “no entendí” o “podés repetirlo”, entonces hay que revisar los prompts o los entrenamientos. Ese tipo de respuestas muestra que el bot no está comprendiendo bien los pedidos, y puede frustrar al usuario (tasa de fallback).

Otro punto clave para mí sería ver cuántas conversaciones terminan en algo concreto, como mostrar información de un auto, una simulación de financiamiento o un contacto con un asesor. Si el bot conversa pero no convierte, no está cumpliendo su rol comercial (conversion rate).

Además, trataría de medir de alguna forma qué tan satisfechos se sienten los usuarios después de hablar con el bot. Puede ser con una típica encuesta al final o con algún indicador indirecto, como si cierran la conversación bien o si vuelven a usarlo.

También me fijaría en cuánto duran las conversaciones y qué tan rápido responde. No quisiera que la gente esté esperando ni que la charla se haga eterna. Idealmente, las respuestas tienen que ser útiles, concisas y al punto.

Y por último, miraría cuánta gente abandona la conversación antes de llegar a nada (retention). Si muchos se van al toque, probablemente haya algo en el flujo que no engancha o que confunde.

Regresión del agente

Para la evaluación de agentes, podríamos definir escenarios clave simulados que representen las distintas etapas del funnel comercial, por ejemplo:

- “Quiero comprar un auto usado de menos de 30.000 km.”
- “Busco un Chevrolet Onix 2020 con menos de 60k km.”
- “¿Puedo financiar mi auto?”
- “¿Qué documentación requiero para financiar?”

Para cada uno de estos casos, se especificará:

- Input inicial del usuario.
- Goal deseado (por ejemplo, mostrar un auto relevante a la búsqueda del usuario).

- Métricas de éxito: ¿Respondió correctamente? ¿Derivó a un humano? ¿Recomendó un auto equivocado?

La evaluación en sí se podría automatizar con el módulo **kavak_agent_evaluator**, que implementé:

- Lee los escenarios desde un archivo JSON (test_cases.json).
- Envía cada caso a un LLM especializado (vía prompt) que comprueba si el agente cumple los objetivos definidos.
- Devuelve un informe con el resultado de cada métrica.

Integrando esto en el CI/CD, podemos detectar regresiones funcionales: si tras un nuevo despliegue el bot deja de resolver correctamente alguno de estos casos (y antes sí), el pipeline falla y nos alerta de la regresión.